

Cada pregunta tiene cuatro posibles respuestas y solo una de ellas es correcta. Para cada pregunta, salvo que el enunciado de ésta indique explícitamente lo contrario, indique la respuesta VERDADERA.

1. En relación con la verificación y validación, se puede afirmar que:

- a. Las actividades de validación son aquellas que buscan asegurar que el software implementa correctamente una función específica.
- b. Las actividades de verificación son aquellas que buscan asegurar que el software construido se corresponde con los requisitos del cliente.
- c. Las actividades de validación son aquellas encaminadas a realizar un tratamiento de excepciones adecuado.
- d. Ninguna de las otras respuestas es verdadera.**

2. Un GIP (grupo independiente de prueba) va a empezar a planificar los casos de prueba de una librería publicada en un repositorio de GitHub con una licencia abierta, la cual se tiene intención de utilizar posteriormente en un proyecto. El repositorio de GitHub, además de incluir todo el código fuente, tiene una Wiki que incluye información sobre la librería, incluyendo detalles sobre el comportamiento esperado de su funcionalidad. ¿Qué pruebas realizará el GIP?

- a. Pruebas de caja blanca, ya que son las únicas que puede realizar.
- b. Pruebas de caja negra, ya que son las únicas que puede realizar.
- c. Podría realizar tanto pruebas de caja blanca como de caja negra, ya que se complementan.**
- d. Aunque podría realizar tanto pruebas de caja blanca como de caja negra, realizará únicamente pruebas de caja blanca ya que si estas pruebas pueden ser realizadas no tiene sentido realizar pruebas de caja negra.

3. Se ha implementado una prueba con JUnit a fin de probar la clase “Calculadora”. A continuación, se muestra el código de esta clase y de esta prueba:

```
public class Calculadora {
    public double operacionSimple(double arg1, String op, double arg2){
        if(op.equals("+")) {
            return arg1+arg2;
        } else if(op.equals("-")) {
            return arg1-arg2;
        } else {
            throw new IllegalArgumentException();
        }
    }
}
```

```
public class CalculadoraTest {
    private Calculadora calculadora;
    @Before
    public void setUp() throws Exception {
        calculadora = new Calculadora();
    }
    @Test
    public void test() {
        double r = calculadora.operacionSimple(3, "+", 1);
        Assert.assertEquals(r, 4, 0);
    }
}
```

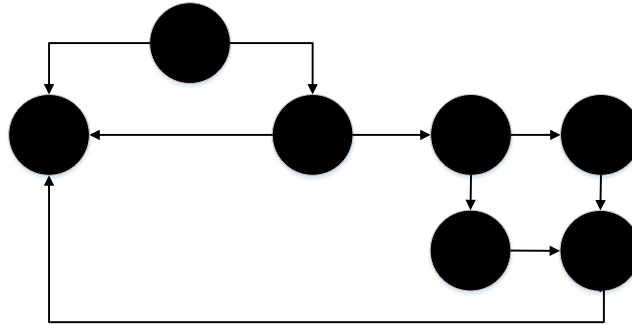
¿Qué se puede afirmar sobre la prueba implementada?

- a. Es una prueba unitaria que podría haberse definido empleando tanto una técnica de caja blanca como de caja negra.**
- b. Tiene que ser necesariamente una prueba definida mediante la técnica de análisis de valores límite.
- c. Tiene que ser necesariamente una prueba definida mediante la técnica de particiones de equivalencia.
- d. Tiene que ser necesariamente una prueba definida mediante la técnica de prueba de la ruta base.

4. El hecho de que un grafo de flujo construido mediante la prueba de la ruta base tenga una complejidad ciclomática de 4 indica:

- a. Que el conjunto base está formado por 4 rutas independientes.
- b. Que existen 4 nodos predicado en el grafo.
- c. Que el grafo tiene 3 regiones.
- d. Que se necesitan definir 8 casos de prueba para cubrir todo el código.

5. Dado el siguiente grafo de flujo construido mediante la prueba de la ruta base, ¿cuántos casos de prueba serán necesarios para cubrir todas las rutas independientes del conjunto base?

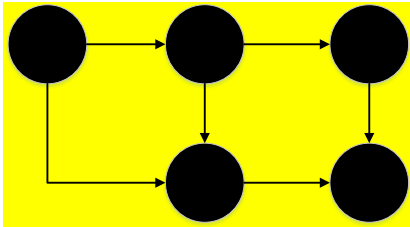


- a. 2.
- b. 3.
- c. 4.
- d. 5.

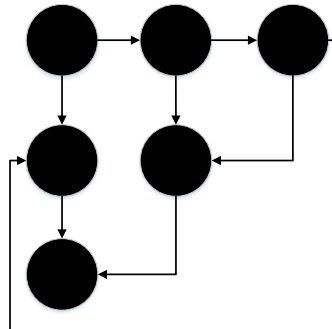
6. ¿Cuál sería el grafo de flujo construido mediante la prueba de la ruta base para el siguiente código fuente?

```
public int test(boolean C1, boolean C2) {
    int a; if (C1 && C2) { a=1; } else { a=2; } return a;
}
```

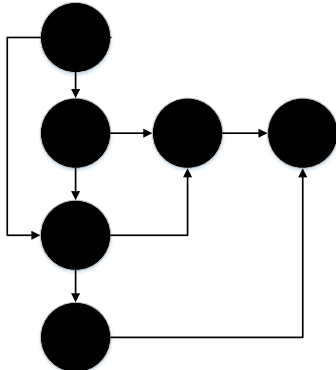
a.



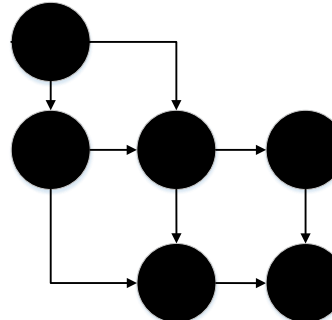
b.



c.



d.



7. Se pretenden realizar pruebas de caja negra utilizando la técnica de particiones de equivalencia para probar una función que recibe como entrada un número que puede ir de 0 a 12000. ¿Qué casos de prueba se podrían definir de acuerdo a esta técnica?
- CP1 (5000), CP2 (15000).
 - CP1 (5000), CP2 (-1000), CP3 (15000).**
 - CP1 (-1000), CP2 (15000).
 - CP1 (-1), CP2 (0), CP3 (12000), CP4 (12001).
8. Se pretenden realizar pruebas de caja negra, utilizando tanto la técnica de particiones de equivalencia como la de análisis de valores límite, para probar una función que recibe como entrada una cadena de caracteres cuya longitud debe estar entre 4 y 6. ¿Qué casos de prueba se podrían definir de acuerdo a estas técnicas?
- CP1 ("ABCD"), CP2 ("AB"), CP3 ("ABCDEFGH").
 - CP1 ("ABCDE"), CP2 ("ABCDEF"), CP3 ("ABC"), CP4 ("ABCDEFG").
 - CP1 ("ABCD"), CP2 ("ABCDEF"), CP3 ("ABC"), CP4 ("ABCDEFG").**
 - CP1 ("ABCD"), CP2 ("ABCDE"), CP3 ("ABCDEF").
9. Se pretenden realizar pruebas de caja negra utilizando la técnica de particiones de equivalencia para probar una función cuyos datos de entrada son un número mayor que cero y un rol que debe tomar alguno de los siguientes valores: "Alumno", "PDI" o "PAS". ¿Qué casos de prueba se podrían definir de acuerdo a esta técnica?
- CP1 (5, "Alumno"), CP2 (5, "PDI"), CP3 (5, "PAS").
 - CP1 (5, "Alumno"), CP2 (5, "Director"), CP3 (-1, "PDI").
 - CP1 (5, "Alumno"), CP2 (5, "PDI"), CP3 (5, "PAS"), CP4 (-1, "Director").
 - CP1 (5, "Alumno"), CP2 (5, "PDI"), CP3 (5, "PAS"), CP4 (-1, "PDI"), CP5 (1, "Director").**
10. Tras implementar un sistema, un programador desea realizar una serie de pruebas de caja negra. Para un método en particular, el programador define las siguientes tablas tras aplicar la técnica de particiones de equivalencia:

Condición de entrada	Clases de equivalencia válidas	Clases de equivalencia inválidas
X	v1	n1, n2
Y	v2	n3, n4
E	v3, v4, v5, v6	n5
K	v7	n6

Caso de prueba	Clases de equivalencia
CP1	v1 v2 v4 v7
CP2	v1 v2 v6 v7
CP3	v1 n3 v3 v7
CP4	v1 n4 v3 v7
CP5	n1 v2 v3 v7
CP6	n2 v2 v3 v7
CP7	v1 v2 n5 v7
CP8	v1 v2 v5 v7

- ¿Se han definido todos los casos de prueba necesarios de acuerdo a la técnica de particiones de equivalencia?
- Si.
 - No, faltaría un caso de prueba por añadir.
 - No, faltarían dos casos de prueba por añadir.**
 - No, faltarían tres casos de prueba por añadir.

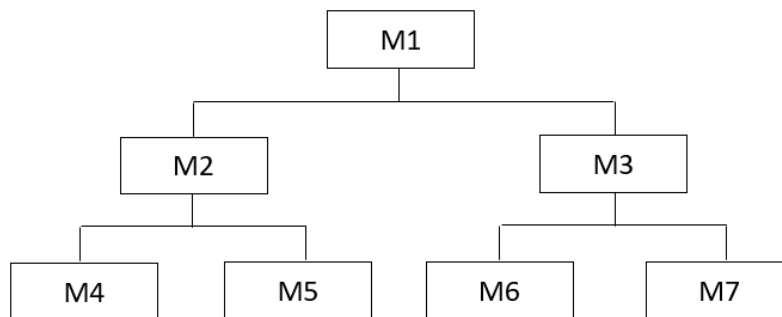
11. Se pretenden realizar pruebas de caja negra empleando la técnica de particiones de equivalencia para probar el siguiente método:

```
public void setValores(String id, double area) { ... }
```

Teniendo en cuenta que “id” debe ser una cadena de texto no vacía de longitud máxima 3 y que “área” tiene que tener valores positivos estrictamente menores que 100, se puede afirmar que:

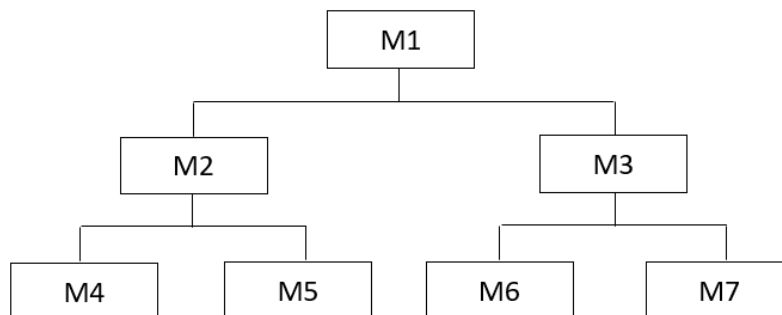
- Sería necesario conocer la implementación del método “setValores” para determinar las clases de equivalencia a las que darían lugar sus parámetros.
- El parámetro “id” daría lugar a 2 clases de equivalencia.
- Habría que definir un total de 6 casos de prueba, uno por cada clase de equivalencia.
- Los casos de prueba CP1(“A5”,100) y CP2 (“B1”,125) serían equivalentes, puesto que estarían probando las mismas clases de equivalencia.

12. Se pretenden realizar las pruebas de integración de un sistema software que tiene los módulos mostrados en la figura adjunta, los cuales están organizados jerárquicamente. Indique la afirmación verdadera considerando que se va a llevar a cabo una estrategia de integración descendente en profundidad:



- Para probar la integración de M1 y M2 resulta imprescindible crear, entre otros, los esqueletos de M4 y M5.
- Para probar la integración de M2, M4 y M5 resulta imprescindible crear un controlador que simule el comportamiento de M1.
- Para probar la integración de M1 y M2 se han de crear, exclusivamente, los esqueletos de M4 y M5.
- Una vez probada la integración de M1 y M2, el siguiente paso consiste en reemplazar el esqueleto de M3 por el módulo M3.

13. Se pretenden realizar las pruebas de integración de un sistema software que tiene los módulos mostrados en la figura adjunta, los cuales están organizados jerárquicamente. Indique la afirmación verdadera considerando que se va a llevar a cabo una estrategia de integración ascendente:



- Para probar la integración de M1 y M2, resulta recomendable que previamente se haya probado la integración de M2, M4 y M5.
- Una vez probada la integración de M6 y M7, el siguiente paso consiste en probar la integración de M1 y M3.
- Para probar la integración de M4 y M5 resulta imprescindible crear el esqueleto de M2.
- Hay que crear un controlador que simule el comportamiento de M2 para probar la integración de M4 y M5.

14. ¿En qué tipo de pruebas se provocan situaciones límite para que el software falle?

- a. Pruebas de seguridad.
- b. Pruebas de estrés/tensión.**
- c. Pruebas de rendimiento.
- d. Pruebas de regresión.

15. Tras finalizar la implementación de una determinada aplicación software, un equipo de desarrollo ha buscado y reunido a un conjunto de posibles usuarios finales a los que ha organizado en tres grupos. Dichos usuarios han probado la aplicación software empleando para ello unos ordenadores puestos a su disposición por el equipo de desarrollo que han sido configurados por este equipo específicamente para la realización de las pruebas. Durante las pruebas, cada grupo de usuarios utilizó la aplicación para ejecutar diferentes casos de uso y terminó rellenando una encuesta, la cual fue posteriormente procesada por el equipo de desarrollo a fin de mejorar la aplicación software implementada. ¿Qué tipo de pruebas se han realizado?

- a. Pruebas de humo.
- b. Pruebas de despliegue.
- c. Pruebas alfa.**
- d. Pruebas beta.